

P3642R0

Carry-less product: `std::clmul`

Published Proposal, 2025-03-13

This version:

<https://eisenwave.github.io/cpp-proposals/clmul.html>

Author:

[Jan Schultke](#)

Audience:

SG6, SG18

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](#)

Abstract

Add widening and non-widening carry-less multiplication functions.

Table of Contents

1 Introduction

2 Motivation

2.1 Parity computation

2.2 Fast space-filling curves

3 Possible implementation

3.1 Hardware support

4 Design Considerations

4.1 Naming

4.2 SIMD

5 Proposed wording

References

Normative References

Informative References

§ 1. Introduction

Carry-less multiplication is a simple numerical operation on unsigned integers. It can be seen as a regular multiplication where `xor` is being used as a reduction instead of `+`.

It is also known as "XOR multiplication" and "polynomial multiplication". The latter name is used because mathematically, it is equivalent to performing a multiplication of two polynomials in GF(2), where each bit is a coefficient.

I propose a `std::clmul` function to perform this operation:

```
template<unsigned_integral T> // constraint is exposition-only
constexpr T clmul(T x, T y) noexcept;
```

I also propose a widening operation in the style of [\[P3161R2\]](#), as follows:

```
template<class T>
struct mul_wide_result {
    T low_bits;
    T high_bits;
};

template<unsigned_integral T>
constexpr mul_wide_result<T> clmul_wide(T x, T y) noexcept;
```

§ 2. Motivation

Carry-less multiplication is an important operation in a number of use cases:

- **CRC Computation:** While cyclic redundancy checks can theoretically be performed with a finite field of any length, in practice, $\text{GF}(2)[X]$, the *polynomial ring* over the *Galois field* with two elements is used. Polynomial addition in this ring can be implemented via `xor`, and multiplication via `clmul`, which makes cyclic redundancy checks considerably faster.
- **Cryptography:** `clmul` may be used to implement AES-GCM. [Intel1] describes this process in great detail and motivates hardware support for carry-less multiplication via the `pclmulqdq` instruction.
- **Bit manipulation:** `clmul` performs a large amount of `<<` and `xor` operations in parallel. This is utilized in the reference implementation [Schultke1] of `std::bit_compressor`, proposed in [P3104R3]. For example, the form `clmul(x, -1u)` computes the bitwise inclusive parity for each bit of `x` and the bits to its right.

Carry-less multiplication is of such great utility that there is widespread hardware support, some dating back more than a decade. See below for motivating examples.

§ 2.1. Parity computation

The parity of an integer `x` is `0` if the number of one-bits is even, and `1` if it is odd. It is also equivalent to `popcount(x) & 1`.

EXAMPLE 1

The special form `clmul(x, -1)` computes the parity of each bit in `x` and the bits to its right. The most significant bit holds the parity of `x` as a whole.

```
bool parity(uint32_t x) {
2   return std::clmul(x, -1u) >> 31;
}
```

§ 2.2. Fast space-filling curves

The special form `clmul(x, -1)` can be used to accelerate the computation of Hilbert curves. To properly understand this example, I will explain the basic notion of space-filling curves.

We can fill space using a 2D curve by mapping the index `i` on the curve onto Cartesian coordinates `x` and `y`. A naive curve that fills a 4x4 square can be computed as follows:

```
struct pos { uint32_t x, y; };  
  
pos naive_curve(uint32_t i) { return { i % 4, i / 4 }; }
```

When mapping the index `i = 0, 1, ..., 0xf` onto the returned 2D coordinates, we obtain the following pattern:

```
0 1 2 3  
4 5 6 7  
8 9 a b  
c d e f
```

The problem with such a naive curve is that adjacent indices can be positioned very far apart (the distance increases with row length). For image processing, if we store pixels in this pattern, cache locality is bad; two adjacent pixels can be very far apart in memory.

A [Hilbert curve](#) is a family of space-filling curves where the distance between two adjacent elements is 1:

```
0 1 e f  
3 2 d c  
4 7 8 b  
5 6 9 a
```

De-interleaving bits of `i` into `x` and `y` yields a [Z-order curve](#), and performing further transformations yields a [Hilbert curve](#).

EXAMPLE 2

`clmul` can be used to compute the bitwise parity for each bit and the bits to its right, which is helpful for computing Hilbert curves. Note that the following example uses the `std::bit_compress` function from [P3104R3], which may also be accelerated using `std::clmul`.

```
pos hilbert_to_xy(uint32_t i)
{
    // De-interleave the bits of i.
    uint32_t i0 = std::bit_compress(i, 0x55555555u); // abcdefgh -> b
    uint32_t i1 = std::bit_compress(i, 0xaaaaaaaau); // abcdefgh -> a

    // Undo the permutation that Hilbert curves apply on top of Z-order
    uint32_t A = i0 & i1;
    uint32_t B = i0 ^ i1 ^ 0xffffu;
10    uint32_t C = std::clmul(A, -1u) >> 16;
11    uint32_t D = std::clmul(B, -1u) >> 16;

    uint32_t a = C ^ (i0 & D);
    return { .x = a ^ i1, .y = a ^ i0 ^ i1 };
}
```

This specific example is taken from [rawrunprotected1]. [Warren1] explains the basis behind this computation of Hilbert curves using bitwise operations.

When working with space-filling curves, the inverse operation is also common: mapping the Cartesian coordinates onto an index on the curve. In the case of Z-order curves aka. Morton curves, this can be done by simply interleaving the bits of `x` and `y`. A Z-order curve is laid out as follows:

```
0 1 4 5
2 3 6 7
8 9 c d
a b e f
```

EXAMPLE 3

`clmul` can be used to implement bit-interleaving in order to generate a [Z-order curves](#).

```
uint32_t xy_to_morton(uint32_t x, uint32_t y)
{
3   uint32_t lo = std::clmul(x, x) << 0; // abcd -> 0a0b0c0d
4   uint32_t hi = std::clmul(y, y) << 1; // abcd -> a0b0c0d0
   return hi | lo;
}
```

NOTE: In the example above, `std::clmul(x, x)` is equivalent to [\[P3104R3\]](#)'s `std::bit_expand(x, 0x55555555u)`.

§ 3. Possible implementation

A naive implementation looks as follows:

```
template<unsigned_integral T>
constexpr T clmul(const T x, const T y) noexcept
{
    T result = 0;
    for (int i = 0; i < numeric_limits<T>::digits; ++i) {
        result ^= (x << i) * ((y >> i) & 1);
    }
    return result;
}
```

§ 3.1. Hardware support

The implementation difficulty lies mostly in utilizing available hardware instructions, not in the naive fallback implementation.

In the following table, let `uN` denote `N`-bit unsigned integer operands, and `xN` denote the amount of operands that are processed in parallel.

Operation	x86_64	ARM	RV64
-----------	--------	-----	------

<code>clmul u64 -> u128</code>	<code>pclmulqdq</code> ^{PCLMULQDQ} <code>pmull+pmull2</code> ^{Neon} <code>clmul+clmulh</code> ^{Zbc, Zbkc}
<code>clmul u64 -> u64</code>	<code>pmull</code> ^{Neon} <code>clmul</code> ^{Zbc, Zbkc}
<code>clmul u8x8 -> u16x8</code>	<code>pmull</code> ^{Neon}
<code>clmul u8x8 -> u8x8</code>	<code>pmul</code> ^{Neon}

EXAMPLE 4

A limited x86_64 implementation of `clmul_wide` may look as follows:

```
#include <immintrin.h>
#include <cstdint>

mul_wide_result<uint64_t> clmul_wide(uint64_t x, uint64_t y) noexcept
{
    __m128i x_128 = _mm_set_epi64x(0, x);
    __m128i y_128 = _mm_set_epi64x(0, y);
    __m128i result_128 = _mm_clmulepi64_si128(x_128, y_128, 0);
    return {
        .low_bits = uint64_t(_mm_extract_epi64(result_128, 0)),
        .high_bits = uint64_t(_mm_extract_epi64(result_128, 1))
    };
}
```

§ 4. Design Considerations

Multiple design choices lean on [\[P0543R3\]](#) and [\[P3161R2\]](#). Specifically,

- the choice of header `<numeric>`,
- the choice to have a widening operation,
- the `_wide` naming scheme,
- the `mul_wide_result` template, and
- the decision to have a `(T, T)` parameter list.

§ 4.1. Naming

Carry-less multiplication is also commonly called "Galois Field Multiplication" or "Polynomial Multiplication".

The name `clmul` was chosen because it carries no domain-specific connotation, and because it is widespread:

- Intel refers to `PCLMULQDQ` As "Carry-Less Multiplication Quadword" in its manual; see [\[Intel2\]](#).
- RISC-V refers to `clmul` as carry-less multiplication, and this is obvious from the mnemonic.
- The Wikipedia article for this operation is titled "Carry-less product" [\[Wikipedia1\]](#).

§ 4.2. SIMD

Many but not all use cases of carry-less multiplication operate on large blocks of data, and SIMD is ideal for that purpose. I do not yet propose carry-less multiplication for `std::simd`, but a future paper, or perhaps this paper could do so, if LEWG wants that.

NOTE: Only ARM supports eight 8-bit multiplications in parallel. Other than that, hardware only supports scalar multiplications, albeit using vector registers except for RISC-V; see [§ 3.1 Hardware support](#).

§ 5. Proposed wording

The proposed changes are relative to the working draft of the standard as of [\[N5001\]](#), with the changes in [\[P3161R2\]](#) applied.

Update subclause [\[version.syn\]](#) paragraph 2 as follows:

```
#define __cpp_lib_clamp                201603L // also in <algorithm>
#define __cpp_lib_clmul                20????L // also in <algorithm>
[...]
```

`#define __cpp_lib_overflow_arithmetic` ~~20????L~~ 20????L // a

In subclause [\[numeric.ops.overview\]](#), update the synopsis as follows:

```
template<class T, class U>
    constexpr T saturate_cast(U x) noexcept;           // freestanding

template<class T>
struct add_carry_result {                             // freestanding
    T low_bits;
    bool overflow;
};

template<class T>
using sub_borrow_result = add_carry_result;           // freestanding

template<class T>
struct mul_wide_result {                             // freestanding
    T low_bits;
    T high_bits;
};

template<class T>
struct div_result {                                   // freestanding
    T quotient;
    T remainder;
};

template<class T>
    constexpr add_carry_result<T>
        add_carry(T x, T y, bool carry) noexcept;    // freestanding
template<class T>
    constexpr sub_borrow_result<T>
        sub_borrow(T left, T right, bool borrow) noexcept; // freestanding
template<class T>
    constexpr mul_wide_result<T>
        mul_wide(T x, T y) noexcept;                 // freestanding
template<class T>
    constexpr mul_wide_result<T>
        clmul_wide(T x, T y) noexcept;               // freestanding
template<class T>
    constexpr div_result<T>
        div_wide(T dividend_high, T dividend_low,    // freestanding
                T divisor ) noexcept;
```

```

template<class T>
constexpr bool
    is_div_defined(T dividend, T divisor) noexcept           // freestanding
template<class T>
constexpr bool
    is_div_wide_defined(T dividend_high,                      // freestanding
                        T dividend_low,
                        T divisor) noexcept;

// [numeric.clmul], carry-less product
template<class T>
constexpr T clmul(T x, U y) noexcept;                       // freestanding

```

In subclause [\[numeric.overflow\]](#) (known as [\[numeric.sat\]](#) at the time of writing), insert the following, immediately following the description of [mul_wide](#):

```

template<class T>
constexpr mul_wide_result<T> clmul_wide(T x, T y) noexcept;

```

Let $\oplus$ denote the exclusive bitwise OR operation (`[expr.xor]`). Let x_n and y_n denote the n^{th} least significant bit in the base-2 representation of x and y respectively. Let w be the width of T .

Constraints: T is an unsigned integer type (`[basic.fundamental]`).

Returns: Let c be an integer, where the value of the n^{th} least significant bit in the base-2 representation of c , c_n , is given by Formula ?? below. The result object is initialized so that

- the bits of `low_bits` are the w least significant bits of c , and
- the bits of `high_bits` are subsequent bits of c .

$$c_n = \bigoplus_{i=0}^n x_i y_{n-i}$$

NOTE: Italicized text such as c shall be in math font.

In subclause [\[numeric.ops\]](#), append a subclause immediately following [\[numeric.overflow\]](#) (known as [\[numeric.sat\]](#) at the time of writing):

Carry-less product [numeric.clmul]

```
template<class T>
constexpr T clmul(T x, T y) noexcept;
```

Constraints: T is an unsigned integer type ([basic.fundamental]).

Returns: `clmul_wide(x, y).low_bits` ([numeric.overflow]).

§ References

§ Normative References

[N5001]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 17 December 2024. URL: <https://wg21.link/n5001>

[P3161R2]

Tiago Freire. *Unified integer overflow arithmetic*. 15 July 2024. URL: <https://wg21.link/p3161r2>

§ Informative References

[Intel1]

Shay Gueron; Michael E. Kounavis. *Intel® Carry-Less Multiplication Instruction and its Usage for Computing the GCM Mode*. URL: <https://www.intel.com/content/dam/develop/external/us/en/documents/clmul-wp-rev-2-02-2014-04-20.pdf>

[Intel2]

Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*. URL: <https://software.intel.com/en-us/download/intel-64-and-ia-32-architectures-sdm-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c-3d-and-4>

[P0543R3]

Jens Maurer. *Saturation arithmetic*. 19 July 2023. URL: <https://wg21.link/p0543r3>

[P3104R3]

Jan Schultke. *P3104R3 Bit permutations*. URL: <https://wg21.link/p3104r3>

[RAWRUNPROTECTED1]

rawrunprotected. *2D Hilbert curves in $O(1)$* . URL: <http://threadlocalmutex.com/?p=188>

[Schultke1]

Jan Schultke. *C++26 Bit permutations reference implementation*. URL:
<https://github.com/Eisenwave/cxx26-bit-permutations>

[Warren1]

Henry S. Warren, Jr. *Hacker's Delight, Second Edition*. URL:
<https://doc.lagout.org/security/Hackers%20Delight.pdf>

[Wikipedia1]

Wikipedia community. *Carry-less product*. URL: https://en.wikipedia.org/wiki/Carry-less_product